Air Temperature Pipeline

Introduction

waterlogged is a data processing package for water quality data from continuous data loggers. The goal of water-logged is to streamline data processing via the compilation of raw data files, application of standardized quality control protocols and export of clean data files for analysis or publication. All quality control processes are documented via logs and not require proprietary software.

This tutorial will guide users through processing continuous air temperature data from TidbiT loggers.

Setup

This vignette assumes you have already installed `waterlogged`:

```
install.packages("pak")
pak::pkg_install("dfo-pacific-science/waterlogged@main")
```

```
library(waterlogged)
```

Prepare Raw Data

Often there will be multiple data files, either files from multiple loggers or files downloaded from a single logger at different times. These files need to be bound together to create a continuous data set for each logger.

In the case loggers were activated prior to deployment or were removed before downloading, the data will need to be trimmed to the deployment and removal dates.

This step requires:

1. Folder containing raw .csv files exported from HOBOware
    - Note that the raw data folder must contain ONLY raw data files of the same metric (e.g. all air-temperature files).
2. A .csv of metadata (*metadata_file*) containing:
    - *sn* serial number of logger
    - *site_station_code* A unique ID for the station where the logger was deployed
    - *timestamp_deployed* timestamp of logger deployment
    - *timestamp_removed* timestamp of logger removal

Define paths to the test data.

```
metadata_file  <- system.file("extdata/metadata/metadata.csv",
                              package = "waterlogged")
airtemp_folder    <- system.file("extdata/water_temperature/airtemp/raw",
                                 package = "waterlogged")
log_root_folder <- tempdir()
```

The function **bind_hobo_files** will load all .csv files in the *path_to_raw_folder*, match the serial number in the logger file with the station name and serial number in the *metadata* file, remove data recorded before the deployment and after the removal timestamps, and bind data files from the same station.

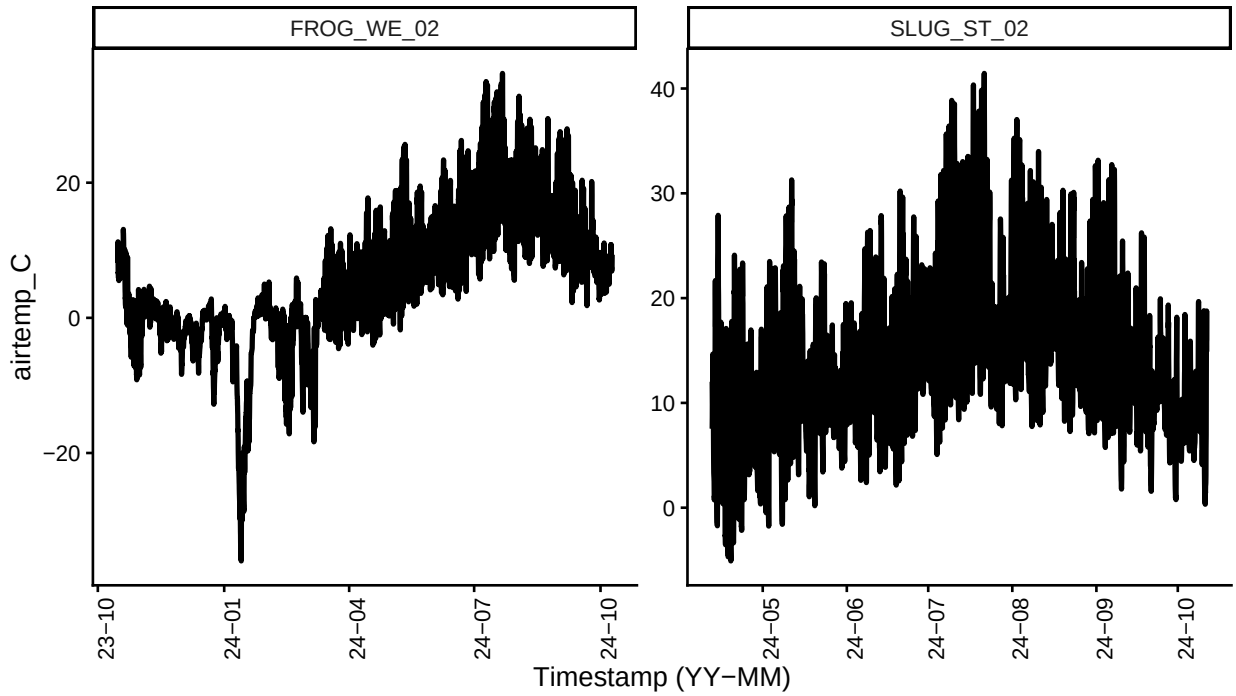
To save bound files, specify the path to the desired output folder with the optional variable *path_to_output_folder*. Saving data from intermediate data processing steps allows users to complete the qaqc process during multiple sessions by using **get_logger_data** to retrieve processed data. Data files will be saved in the designated output folder and organized by station and year. Data files are named using the convention *AT_station_start-date_end-date_v0.1*.

bind_hobo_files will produce a list of two elements. The first is a tibble containing all of the data from the raw folder, trimmed to the deployment period and identified by the site station code matching the logger serial number in the metadata file. The second is a summary plot of the data grouped by site_station_code. The user should ensure all expected logger stations and date ranges are present in the summary plot.

```
at_bound <- bind_hobo_files(
  path_to_raw_folder = airtemp_folder,
  metadata_path  = metadata_file,
)

# save the first element of the list as a dataframe:
at_data <- at_bound[[1]]

# view summary plot:
at_bound[[2]]
```



```r
# summarize the trimmed and bound data
cat("AT data:", nrow(at_data), "rows,",
    length(unique(at_data$site_station_code)), "stations\n")
#> AT data: 13019 rows, 2 stations
```

Individual air temperature data files for each station and year have been saved in their respective folders, and the complete timeseries from all data files saved in the environment as **at_data**.

Automated Quality Control Protocols

qc_airtemp applies an automatic qc function to the raw air temperature data set to remove outliers and flag potentially erroneous data determined by user-modifiable thresholds. All raw data values are preserved in the column *airtemp_C*, **qc_airtemp** creates a new data column *airtemp_C_adj* that contains the reviewed including an adjustments as well as logical flag columns *OUTLIER_AT* and *FLAG_RANGE* that indicate if data has been removed or flagged. The default thresholds are as follows:

- temp_high_flag_C: Values above this value will flagged as FLAG_RANGE in flag_code column. Default is 50°C.
- temp_low_flag_C: Values below this value will be flagged as FLAG_RANGE in flag_code column. Default is -40°C.
- temp_high_limit_C: Values above this value will be set as NA and flagged as OUTLIER_AT. Default is Inf.
- temp_low_limit_C: Values below this value will be set as NA and flagged as OUTLIER_AT. Default is -Inf.

Limits reflect the operating limits of the logger. In this example the loggers deployed at our sites are operational within -20°C and 40°C, we need to remove all temperature values that exceed these limits. We set high and low temperature limits in *qc_airtemp* to remove all data beyond those limits. Based on local knowledge we also do not expect air temperature to drop below -10°C at this location, and so we want data flags to be produced for values beyond this limit so we can examine any data that falls below this value.

The optional input variable *log_root* is the path to the desired folder where a CSV file of all QC flags and adjustments for each station will be saved.

```r
at_qc <- qc_airtemp(
  input_data = at_data,
  temp_high_limit_C = 40,
  temp_low_limit_C = -20,
  temp_low_flag_C = -10,
  log_root = log_root_folder,
  select_station = unique(at_data$site_station_code)[1]
)
```

```
#> QAQC results:
#>   Total rows: 8643
#>   OUTLIER_AT: 114
#>   FLAG_RANGE: 378
```

Note that data labeled as OUTLIER have been removed (114 data points), while the data labeled as FLAG_RANGE have been retained for visual inspection. The QC log produced will look like this:

```
#>
#> QAQC log entries:
#> # A tibble: 6 x 4
#>   action    action_note                                                ts_start            ts_end
#>   <chr>     <chr>                                                      <dttm>              <dttm>
#> 1 AUTO_QAQC Air temperature outside [-20, 40] C; set to NA             2024-01-11 07:00:00 2024-01-14 13
#> 2 AUTO_QAQC Air temperature outside [-20, 40] C; set to NA             2024-01-14 15:00:00 2024-01-15 14
#> 3 AUTO_QAQC Air temperature outside [-20, 40] C; set to NA             2024-01-15 18:00:00 2024-01-16 04
#> 4 AUTO_QAQC Air temperature outside [-10, 50] C; flagged for review    2023-12-24 05:00:00 2023-12-24 11
#> 5 AUTO_QAQC Air temperature outside [-10, 50] C; flagged for review    2023-12-24 17:00:00 2023-12-24 21
#> 6 AUTO_QAQC Air temperature outside [-10, 50] C; flagged for review    2024-03-06 23:00:00 2024-03-07 10
```

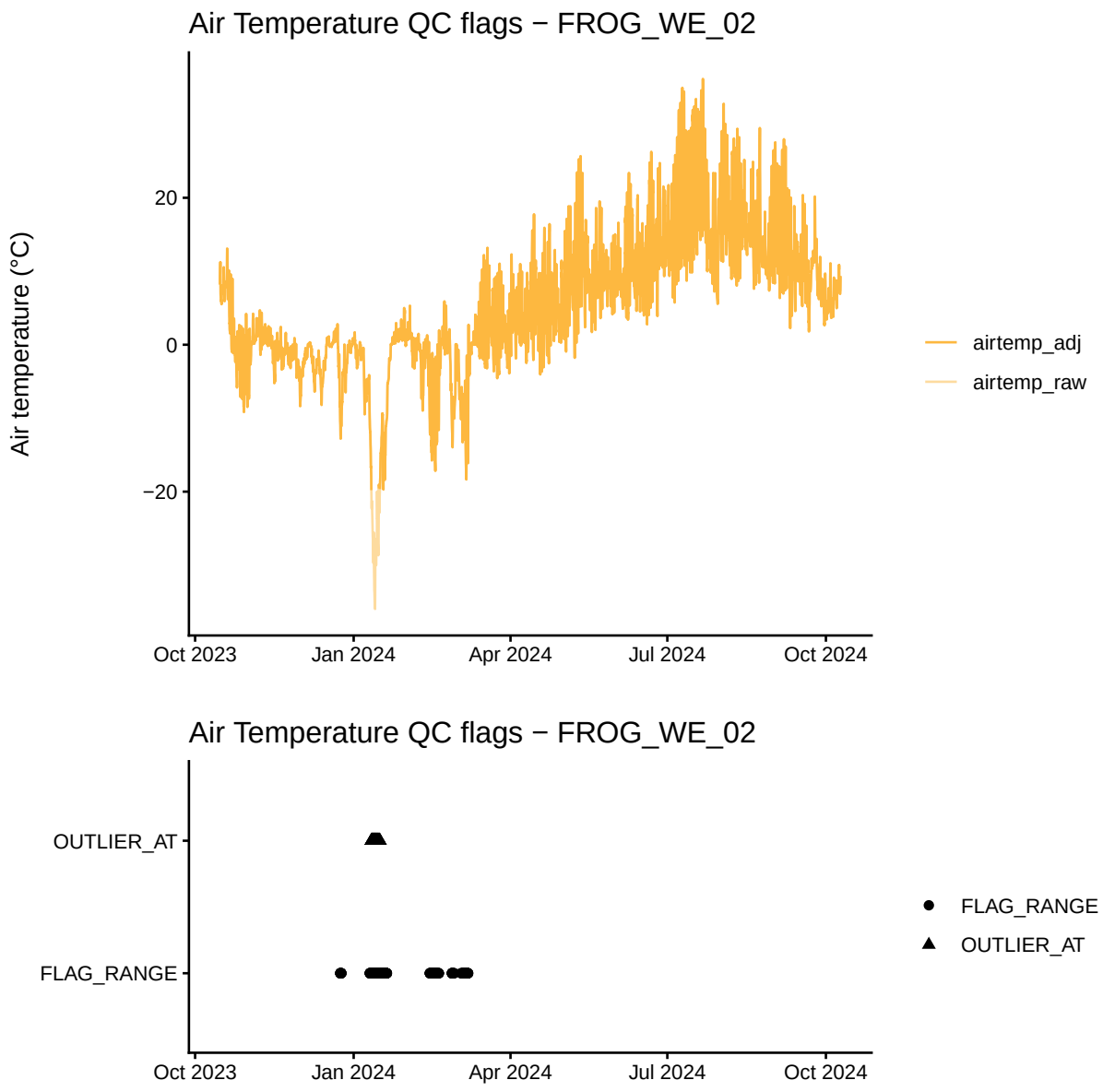
Visual Review

plot_qaqc_timeseries creates an interactive multi-panel plot for reviewing the raw data, data flags and automatic and manual data adjustments.

How to read the plot:

- Light traces = raw values (original, unmodified)
- Dark traces = adjusted values (after auto QAQC and any subsequent manual adjustments)
- Black symbols = flagged data (ice, dry, error)
- Use the range slider at the bottom to zoom into specific periods
- Hover over points for exact values and timestamps

```
plot_qaqc_timeseries(
  airtemp_data      = at_qc,
  select_station = unique(at_data$site_station_code)[1],
  static = TRUE
)
```



The data can be inspected by clicking and dragging over a section of the graph. If the values are reasonable and any data flags that are reviewed do not indicate a logger error the qc process is complete and the data can be saved for publication or analysis using **write_clean_data**. Note that data flags are meant as an indication to inspect the data series for possible errors are not confirmation that the data is incorrect.

If data errors are discovered, data can be manually adjusted with the function *adjust_logger_NA*. In this example, during a site visit to FROG_WE_02 at "2024-07-29 16:00:00", when removing the air temperature logger for inspection, the technician dropped the logger in the stream and could not retrieve it. The next day, the logger was found and re-installed at "2024-07-30 9:45:00". This "disturbance" event was recorded in field notes. Since the logger was recording water temperature, not air temperature, during that time period, those data should be removed.

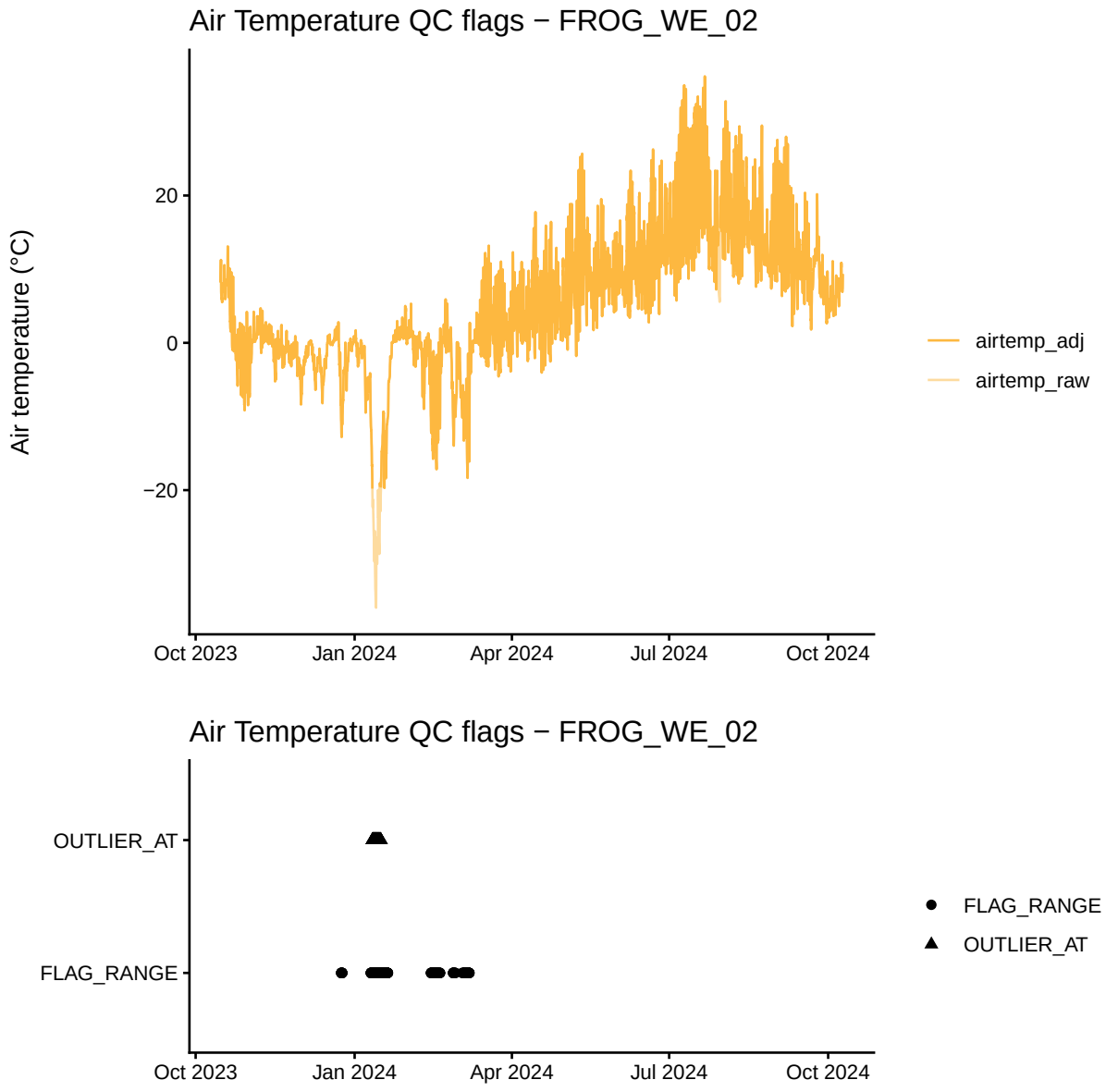
If we zoom in on that section of the timeseries, we can see a small dip in temperature. This isn't flagged as an outlier, as it is within the defined temperature range. This disturbance event can be removed manually, as follows:

```
at_qc2 <- adjust_logger_NA(
  input_data       = at_qc,
  select_station   = unique(at_data$site_station_code)[1],
  metric           = "airtemp",
  reason_to_adjust = "disturbance",
  timestamp_start  = "2024-07-29 16:00:00",
  timestamp_end    = "2024-07-30 09:45:00",
  manual_note      = "Logger dropped in stream - reinstalled next day",
  log_root = log_root_folder
)
```

Final Review

These changes can be reviewed by re-running the plot function. Zoom in on the disturbance time period to make sure the desired values were removed. The adjusted traces should look reasonable with no obvious artifacts. Compare raw (light) vs adjusted (dark) to see exactly what changed.

```
plot_qaqc_timeseries(
  airtemp_data      = at_qc2,
  select_station = "FROG_WE_02",
    static=TRUE
)
```



Once all data are reviewed and adjustments are complete, clean data is saved using **write_clean_data**.

Export Clean Data (v1.0)

Writes the final cleaned data to CSV files, one per station per year.

What gets exported:

- `site_station_code`, `timestamp` (identifiers)
- `airtemp_C`, `airtemp_C_adj` (raw and adjusted air temperature)
- All `FLAG_*`, `OUTLIER_*` columns (the full audit trail)

```r
cat("Final dataset:", nrow(at_qc), "rows x", ncol(at_qc), "columns\n\n")
#> Final dataset: 8643 rows x 9 columns

cat("\nSummary of raw AT (degrees C):\n")
#>
#> Summary of raw AT (degrees C):
print(summary(at_qc2$airtemp_C))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> -35.970  -0.704   4.168   5.027  11.175  36.173

cat("\nSummary of adjusted AT (degrees C):\n")
#>
#> Summary of adjusted AT (degrees C):
print(summary(at_qc2$airtemp_C_adj))
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#> -19.979  -0.535   4.350   5.449  11.297  36.173     132
```

```r
write_clean_data(
  input_data            = at_qc2,
  path_to_output_folder = output,
  metric                = "airtemp"
)
```